

CLI, Terminal & Scripts Shell

Maîtriser la ligne de commande pour le développement logiciel

Terminal	Bash	Zsh	Shell
----------	------	-----	-------

Scripts	Pipes	Cron	Variables
---------	-------	------	-----------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Naviguer et manipuler le système de fichiers depuis le terminal
- ✓ Comprendre les variables d'environnement et le PATH
- ✓ Écrire des scripts shell robustes avec conditions, boucles et fonctions
- ✓ Maîtriser les pipes, redirections et les outils de traitement de texte
- ✓ Gérer les processus, les signaux et les tâches en arrière-plan
- ✓ Automatiser des tâches récurrentes avec cron et des scripts de déploiement

Table des matières

Chapitre 1 — Le terminal — fondamentaux

- 1.1 Shell, terminal et console
- 1.2 Navigation dans le système de fichiers
- 1.3 Manipulation de fichiers et dossiers
- 1.4 Permissions et propriétaires

Chapitre 2 — Variables et environnement

- 2.1 Variables shell
- 2.2 Variables d'environnement
- 2.3 Le PATH expliqué
- 2.4 Fichiers de configuration (.zshrc, .bashrc)

Chapitre 3 — Scripts shell

- 3.1 Structure d'un script
- 3.2 Conditions if/elif/else
- 3.3 Boucles for et while
- 3.4 Fonctions

Chapitre 4 — Pipes, redirections et outils

- 4.1 Pipes | et redirections > >>
- 4.2 grep, sed, awk
- 4.3 find, xargs
- 4.4 jq — traiter le JSON

Chapitre 5 — Processus et automatisation

- 5.1 Gestion des processus
- 5.2 Tâches en arrière-plan
- 5.3 cron et crontab

5.4 Scripts de déploiement

Le terminal — fondamentaux

1.1 Shell, terminal et console

Le terminal est l'interface graphique qui affiche une fenêtre de texte. Le shell est le programme qui tourne à l'intérieur et interprète les commandes. Sur macOS, le shell par défaut est Zsh (depuis macOS Catalina). Sur Linux, c'est Bash. Les deux sont très similaires — la grande majorité des scripts et commandes fonctionnent dans les deux.

Comprendre le terminal est une compétence fondamentale pour tout développeur. La ligne de commande offre une précision, une vitesse et des capacités d'automatisation impossibles à atteindre avec une interface graphique. Toutes les opérations de déploiement, de gestion de serveur, de manipulation de données en masse passent par le terminal.

Définition — Shell

Un shell est un interpréteur de commandes — il lit ce que vous tapez, l'interprète, et communique avec le noyau du système d'exploitation pour exécuter les instructions. Bash (Bourne Again Shell) et Zsh (Z Shell) sont les deux shells les plus répandus. Ils partagent la même syntaxe de base mais Zsh offre des fonctionnalités supplémentaires comme l'autocomplétion avancée.

1.2 Navigation dans le système de fichiers

Le système de fichiers est une arborescence. Vous êtes toujours dans un répertoire courant (current working directory). La commande `pwd` (print working directory) affiche où vous êtes. `cd` (change directory) permet de naviguer. `ls` liste le contenu d'un répertoire.

```
# Afficher le répertoire courant
pwd
# /Users/mohamedfomba/Desktop/Dev/social-seller

# Lister les fichiers (format long avec détails)
ls -la
# -l : format long (permissions, taille, date)
# -a : tout afficher (y compris les fichiers cachés .env)

# Naviguer
cd backend # Aller dans le sous-dossier backend
cd .. # Remonter d'un niveau
cd ../.. # Remonter de deux niveaux
```

```
cd ~ # Aller dans le home directory
cd - # Retourner au répertoire précédent
cd ~/Desktop/Dev/social-seller # Chemin absolu

# Raccourcis utiles

ls -lah # Tailles en format humain (K, M, G)
ls *.ts # Lister tous les fichiers .ts
ls -lt # Trier par date de modification
```

1.3 Manipulation de fichiers et dossiers

```
# Créer

touch file.txt # Créer un fichier vide
mkdir mon-dossier # Créer un dossier
mkdir -p a/b/c # Créer des dossiers imbriqués

# Copier

cp file.txt backup.txt # Copier un fichier
cp -r dossier/ backup-dossier/ # Copier un dossier (-r = récursif)

# Déplacer / Renommer

mv ancien.txt nouveau.txt # Renommer
mv file.txt ../autre-dossier/ # Déplacer

# Supprimer

rm file.txt # Supprimer un fichier
rm -rf dossier/ # Supprimer un dossier et son contenu
# ■■ rm -rf est irréversible – pas de corbeille

# Lire un fichier

cat file.txt # Afficher tout le contenu
head -20 file.txt # Afficher les 20 premières lignes
tail -20 file.txt # Afficher les 20 dernières lignes
tail -f logs/app.log # Suivre un fichier en temps réel

# Rechercher dans un fichier

grep 'tenant_id' src/routes/orders.ts
grep -rn 'TODO' src/ # Récursif avec numéros de ligne
grep -i 'error' logs/ # Insensible à la casse
```

1.4 Permissions et propriétaires

Chaque fichier sur Unix a des permissions définissant qui peut le lire (r), l'écrire (w), ou l'exécuter (x). Ces permissions s'appliquent au propriétaire (user), au groupe (group), et aux autres (others). La commande `chmod` modifie les permissions, `chown` change le propriétaire.

```
# Voir les permissions

ls -la

# -rw-r--r-- 1 mohamedfomba staff 1234 Jun 23 10:00 file.txt
# ^ = type (--fichier, d=dossier)
# ^^^ = permissions propriétaire (rw-)
# ^^^ = permissions groupe (r--)
# ^^^ = permissions autres (r--)

# Rendre un script exécutable

chmod +x deploy.sh

./deploy.sh # L'exécuter

# Permissions numériques

chmod 755 script.sh # rwxr-xr-x (exécutable par tous, modifiable par owner)
chmod 600 .env # rw----- (lisible/modifiable par owner seulement)
chmod 644 config.json # rw-r--r-- (lisible par tous, modifiable par owner)
```

Variables et environnement

2.1 Variables shell

Une variable shell stocke une valeur textuelle. Pour assigner, on écrit `NOM=valeur` (sans espaces autour du `=`). Pour lire, on préfixe avec `$`. Les variables shell sont locales au shell courant — elles ne sont pas disponibles dans les processus enfants (scripts, sous-shells).

```
# Assigner une variable

NOM='Mohamed'

AGE=28

API_URL='https://social-seller-production.up.railway.app'

# Lire une variable

echo $NOM # Mohamed

echo ${NOM} # Mohamed (forme avec accolades – plus sûre)

echo "Bonjour $NOM" # Bonjour Mohamed (interpolation dans "...")

echo 'Bonjour $NOM' # Bonjour \NOM (pas d'interpolation dans '...')

# Commande dans une variable

DATE=$(date +%Y-%m-%d) # Capture la sortie d'une commande

echo $DATE # 2026-06-23

# Valeur par défaut si variable non définie

PORT=${PORT:-3000} # PORT = PORT si défini, 3000 sinon

# Variables en lecture seule

readonly MAX_RETRIES=3
```

2.2 Variables d'environnement

Les variables d'environnement (env vars) sont disponibles dans le processus courant ET tous ses processus enfants. On les crée avec `export`. Elles sont la façon standard de passer des secrets et configurations aux applications — c'est ce qu'utilisent Railway, Docker, et tous les hébergeurs modernes.

```
# Créer une variable d'environnement

export DATABASE_URL='postgresql://localhost:5432/mydb'
```



```

export NODE_ENV='production'

# Lister toutes les variables d'environnement
env

env | grep NODE # Filtrer avec grep

printenv NODE_ENV # Afficher une variable spécifique

# Variable pour une seule commande (temporaire)
NODE_ENV=test npm test # NODE_ENV=test uniquement pour npm test

# Charger un fichier .env dans le shell
export $(cat .env.local | grep -v '^#' | xargs)

# ■■■ Ne jamais faire ça avec .env contenant des secrets en prod

# Vérifier qu'une variable est définie
if [ -z "$API_KEY" ]; then
echo 'Erreur : API_KEY non définie'
exit 1
fi

```

2.3 Le PATH expliqué

Le PATH est une variable d'environnement spéciale qui liste les dossiers où le shell cherche les exécutables. Quand vous tapez npm, le shell parcourt chaque dossier du PATH dans l'ordre jusqu'à trouver un fichier exécutable nommé npm. Si aucun n'est trouvé, vous obtenez 'command not found'.

```

# Afficher le PATH actuel

echo $PATH

# /usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:/usr/local/go/bin

# Ajouter un dossier au PATH (temporaire)
export PATH="$PATH:/usr/local/custom/bin"

# Trouver où est installé un outil

which node # /usr/local/bin/node
which npm # /usr/local/bin/npm
which python3 # /usr/bin/python3

# Si 'command not found' après installation

# 1. Trouver où l'outil est installé

# 2. Ajouter ce dossier au PATH dans ~/.zshrc
export PATH="$PATH:/nouveau/dossier"

```

```
source ~/.zshrc # Recharger la configuration
```

2.4 Fichiers de configuration

Chaque fois que vous ouvrez un nouveau terminal, le shell charge un fichier de configuration. Pour Zsh, c'est ~/.zshrc. Pour Bash, c'est ~/.bashrc ou ~/.bash_profile. C'est là qu'on ajoute des exports de PATH permanents, des alias, et des fonctions utilitaires.

```
# ~/.zshrc – exemples de configuration utile

# PATH personnalisé
export PATH="$HOME/.local/bin:$PATH"

# Aliases pratiques
alias ll='ls -lah'
alias gs='git status'
alias gp='git push'
alias ..='cd ..'
alias ...='cd ../..'

# Alias pour le projet
alias ss='cd ~/Desktop/Dev/social-seller'
alias backend='cd ~/Desktop/Dev/social-seller/backend'

# Fonction utilitaire
mkcd() { mkdir -p "$1" && cd "$1"; }

# Usage : mkcd nouveau-dossier

# Recharger .zshrc sans fermer le terminal
source ~/.zshrc
```

Scripts shell

3.1 Structure d'un script

Un script shell est un fichier texte contenant une séquence de commandes. La première ligne (shebang) indique quel interpréteur utiliser. Les bonnes pratiques : set -e pour arrêter le script à la première erreur, set -u pour traiter les variables non définies comme des erreurs, des commentaires clairs, et des messages de log.

```
#!/bin/bash

# deploy.sh – script de déploiement Social Seller

# Usage : ./deploy.sh [production|staging]

set -e # Arrêter si une commande échoue
set -u # Erreur si variable non définie
set -o pipefail # Erreur si une commande dans un pipe échoue

# Couleurs pour les logs
GREEN='\033[0;32m'
RED='\033[0;31m'
NC='\033[0m' # No Color

log() { echo -e "${GREEN}[INFO]${NC} $1"; }
error(){ echo -e "${RED}[ERROR]${NC} $1" >&2; exit 1; }

# Vérifier les arguments
ENV=${1:-staging}

if [[ "$ENV" != "production" && "$ENV" != "staging" ]]; then
    error "Usage: ./deploy.sh [production|staging]"
fi

log "Déploiement vers $ENV..."

# Suite du script...
```

3.2 Conditions if/elif/else

Les conditions en shell utilisent des opérateurs spécifiques pour tester les chaînes, les nombres, et les fichiers. La syntaxe `[[ ]]` est préférée à `[ ]` car elle est plus robuste et supporte les expressions régulières.

```

# Tests sur les chaînes

if [[ "$NODE_ENV" == "production" ]]; then
echo 'Mode production'
elif [[ "$NODE_ENV" == "staging" ]]; then
echo 'Mode staging'
else
echo 'Mode développement'
fi

# Tests sur les fichiers

if [[ -f .env.local ]]; then
echo 'Fichier .env.local existe'
fi

if [[ -d node_modules ]]; then
echo 'node_modules existe'
fi

if [[ ! -f package-lock.json ]]; then
echo 'package-lock.json manquant'
fi

# Tests numériques

COUNT=$(ls src/ | wc -l)

if [[ $COUNT -gt 10 ]]; then
echo "Plus de 10 fichiers dans src/"
fi

# Tests combinés

if [[ -f .env && "$NODE_ENV" == "production" ]]; then
echo 'Prêt pour la production'
fi

```

3.3 Boucles for et while

```

# Boucle sur une liste de valeurs

for env in development staging production; do
echo "Environnement : $env"
done

# Boucle sur les fichiers

```

```

for file in src/**/*.ts; do
echo "Fichier TypeScript : $file"
done

# Boucle sur les lignes d'un fichier
while IFS= read -r line; do
echo "Ligne : $line"
done < liste.txt

# Boucle avec compteur
for i in {1..5}; do
echo "Tentative $i"
curl -s https://api.example.com/health && break
sleep 2
done

# Boucle while avec condition
ATTEMPT=0
while [[ $ATTEMPT -lt 3 ]]; do
npm test && break
ATTEMPT=$((ATTEMPT + 1))
echo "Échec – tentative $ATTEMPT/3"
done

```

3.4 Fonctions

Les fonctions shell permettent de factoriser le code répétitif. Elles acceptent des arguments via \$1, \$2, etc. et peuvent retourner une valeur (code de sortie 0-255) ou afficher un résultat via echo.

```

# Définir une fonction

check_command() {
local cmd=$1

if ! command -v "$cmd" &> /dev/null; then
echo "Erreur : '$cmd' n'est pas installé"
return 1
fi

echo "✓ $cmd trouvé : $(which $cmd)"
}

# Appeler la fonction

```

```
check_command node
check_command npm
check_command git

# Fonction avec valeur de retour via echo
get_version() {
node --version | tr -d 'v'
}

NODE_VERSION=$(get_version)
echo "Node.js version : $NODE_VERSION"

# Fonction de confirmation
confirm() {
read -p "$1 [y/N] " response
[[ "$response" =~ ^[Yy]$ ]]
}

if confirm 'Déployer en production ?'; then
echo 'Déploiement...'
fi
```

Pipes, redirections et outils

4.1 Pipes et redirections

Le pipe `|` connecte la sortie d'une commande à l'entrée de la suivante. C'est la philosophie Unix : des outils simples qui font une seule chose bien, combinés par des pipes pour résoudre des problèmes complexes. Les redirections `>` et `>>` envoient la sortie vers un fichier.

```
# Pipe : envoyer la sortie d'une commande à une autre
ls -la | grep '.ts' # Lister les fichiers .ts
cat server.log | grep ERROR # Filtrer les erreurs
npm test 2>&1 | tee test.log # Afficher ET sauvegarder

# Redirections
echo 'Hello' > file.txt # Créer/écraser file.txt
echo 'World' >> file.txt # Ajouter à la fin de file.txt
npm test > test.log 2>&1 # Stdout et stderr vers test.log
npm test > /dev/null # Ignorer la sortie

# Stdin depuis un fichier
psql database < migration.sql

# Here document (texte multi-lignes)
cat > config.json << 'EOF'
{
  "port": 8080,
  "env": "production"
}
EOF

# Compter les lignes
cat server.log | wc -l
ls src/ | wc -l
```

4.2 grep, sed, awk

Ces trois outils sont les piliers du traitement de texte en shell. grep filtre les lignes, sed transforme le texte, awk traite les colonnes. Ensemble, ils permettent de manipuler des données textuelles avec une puissance remarquable.

```
# grep – filtrer des lignes

grep 'ERROR' server.log # Lignes contenant ERROR
grep -i 'error' server.log # Insensible à la casse
grep -n 'TODO' src/**/*.ts # Avec numéros de ligne
grep -r 'tenant_id' src/ # Récursif
grep -v 'DEBUG' server.log # Exclure les lignes DEBUG
grep -E 'ERROR|WARN' server.log # Regex étendue (OU logique)

# sed – transformer le texte

sed 's/localhost/prod.example.com/g' .env.example > .env
sed -i 's/version: .*/version: 2.0.0/' package.json
sed '/^#/d' .env.example # Supprimer les commentaires
sed '/^$/d' file.txt # Supprimer les lignes vides

# awk – traiter les colonnes

awk '{print $1}' server.log # Afficher la 1ère colonne
awk -F: '{print $1}' /etc/passwd # Séparateur = :
ls -la | awk '{print $5, $9}' # Taille et nom du fichier
awk '/ERROR/ {count++} END {print count}' server.log # Compter
```

4.3 find et xargs

```
# find – rechercher des fichiers

find . -name '*.ts' # Tous les fichiers .ts
find . -name '*.ts' -not -path '*/node_modules/*'
find . -type f -newer package.json # Modifiés après package.json
find . -size +1M # Fichiers > 1MB
find . -name '.env*' # Fichiers commençant par .env

# xargs – passer les résultats comme arguments

find . -name '*.log' | xargs rm # Supprimer tous les logs
find . -name '*.ts' | xargs grep 'TODO' # Chercher TODO dans tous les .ts

# Combinaisons utiles

find . -name 'node_modules' -type d | xargs du -sh

# Afficher la taille de chaque node_modules
```



```
find . -name '*.ts' -newer src/routes/orders.ts  
# Fichiers .ts modifiés après orders.ts
```

4.4 jq — traiter le JSON

jq est un processeur JSON en ligne de commande. Il permet de parser, filtrer et transformer des réponses JSON d'APIs directement dans le terminal. C'est l'outil indispensable quand on travaille avec des APIs REST via cURL.

```
# Installation  
brew install jq  
  
# Formatter le JSON  
curl -s https://api.example.com/products | jq .  
  
# Extraire une valeur  
curl -s https://api.example.com/health | jq '.status'  
# 'ok'  
  
# Accéder à un tableau  
curl -s https://api.example.com/products | jq '.[0]'  
curl -s https://api.example.com/products | jq '.[0].name'  
  
# Filtrer  
curl -s https://api.example.com/products | jq '.[0] | select(.price > 5000)'  
  
# Transformer  
curl -s https://api.example.com/products | jq '.[0] | {id, name, price}'  
  
# Compter  
curl -s https://api.example.com/products | jq 'length'  
  
# Stocker dans une variable  
STATUS=$(curl -s https://api.example.com/health | jq -r '.status')  
# -r = raw (sans guillemets)  
echo $STATUS # ok (sans guillemets)
```

Processus et automatisation

5.1 Gestion des processus

```
# Lister les processus

ps aux # Tous les processus
ps aux | grep node # Processus Node.js
top # Vue dynamique (q pour quitter)
htop # Version améliorée de top

# Trouver le PID d'un processus sur un port
lsof -i :8080 # Qui utilise le port 8080 ?
lsof -i :3000 # Qui utilise le port 3000 ?

# Tuer un processus
kill 1234 # Signal SIGTERM (arrêt propre)
kill -9 1234 # Signal SIGKILL (arrêt forcé)
killall node # Tuer tous les processus node

# Libérer un port
lsof -ti :8080 | xargs kill -9

# Trouver le PID sur le port 8080 et le tuer
```

5.2 Tâches en arrière-plan

```
# Lancer en arrière-plan avec &
npm run dev &
mailpit &
echo "PID: $!" # PID du dernier processus lancé

# Voir les jobs en cours
jobs

# Mettre en avant-plan
fg %1 # Ramener le job 1 en avant-plan

# Envoyer en arrière-plan (Ctrl+Z puis bg)
# 1. Ctrl+Z pour suspendre
```

```
# 2. bg pour reprendre en arrière-plan

# nohup – continuer après fermeture du terminal
nohup npm run dev > dev.log 2>&1 &

# Lancer plusieurs services en parallèle
npm run dev &
mailpit &
wait # Attendre que tous les jobs soient terminés
```

5.3 cron et crontab

cron est le planificateur de tâches Unix. Il exécute automatiquement des commandes selon un calendrier défini dans le crontab. La syntaxe cron est : minute heure jour-du-mois mois jour-de-semaine commande.

```
# Éditer le crontab
crontab -e

# Syntaxe : MIN HEURE JOUR MOIS JOUR-SEMAINE COMMANDE
# * * * * *

# Exemples
0 * * * * /usr/local/bin/backup.sh # Toutes les heures
0 2 * * * /usr/local/bin/cleanup.sh # Tous les jours à 2h
0 9 * * 1 /usr/local/bin/report.sh # Lundi à 9h
*/5 * * * * /usr/local/bin/healthcheck.sh # Toutes les 5 min
0 0 1 * * /usr/local/bin/monthly.sh # 1er de chaque mois

# Voir le crontab actuel
crontab -l

# Supprimer le crontab
crontab -r

# Tester la syntaxe cron
# crontab.guru – éditeur visuel en ligne
```

5.4 Scripts de déploiement

Un script de déploiement automatise toutes les étapes entre le code validé et le service en production. Un bon script vérifie les prérequis, exécute les tests, déploie, vérifie que le déploiement a réussi, et notifie en cas d'erreur.

```
#!/bin/bash

# deploy.sh – déploiement complet Social Seller

set -euo pipefail

GREEN='\033[0;32m'; RED='\033[0;31m'; NC='\033[0m'

log() { echo -e "${GREEN}[$(date +%H:%M:%S)] $1${NC}"; }
error(){ echo -e "${RED}[ERREUR] $1${NC}" >&2; exit 1; }

# 1. Vérifications préalables
log 'Vérification des prérequis...'
command -v git || error 'git non installé'
command -v node || error 'node non installé'
[[ -f .env.local ]] || error '.env.local manquant'

# 2. Tests
log 'Exécution des tests...'
cd backend && npm test || error 'Tests échoués'
cd ..

# 3. Vérification TypeScript
log 'Vérification TypeScript...'
cd backend && npx tsc --noEmit || error 'Erreurs TypeScript'
cd ..

# 4. Commit et push
log 'Push vers GitHub...'
git add -A
git commit -m "deploy: $(date +%Y-%m-%d) - $1" || log 'Rien à commiter'
git push origin main

# 5. Vérification post-déploiement
log 'Attente du déploiement Railway (60s)... '
sleep 60

STATUS=$(curl -s -o /dev/null -w '%{http_code}' \
https://social-seller-production.up.railway.app/health)

if [[ "$STATUS" == "200" ]]; then
log 'Déploiement réussi ✓'
else
error "Health check échoué (HTTP $STATUS)"
```

fi

■ Bonne pratique

Ajouter ce script en alias dans `.zshrc` : `alias deploy='cd ~/Desktop/Dev/social-seller && ./deploy.sh'`. Un déploiement complet se réduit à taper 'deploy' dans le terminal.

■ À retenir

- ✓ Le terminal est indispensable — toute automatisation sérieuse passe par là
- ✓ `export` rend une variable disponible aux processus enfants — obligatoire pour les env vars
- ✓ `set -euo pipefail` en début de script — il stoppe à la première erreur
- ✓ Les pipes | permettent de chaîner des outils simples pour résoudre des problèmes complexes
- ✓ `jq` est indispensable pour traiter les réponses JSON d'APIs en shell
- ✓ Un bon script de déploiement : prérequis → tests → TS check → push → vérification

— ■ Exercices

Exercice 1 — Script d'initialisation projet

Créer un script qui vérifie et configure l'environnement de développement.

- Vérifier que `node ≥ 18`, `npm`, `git` sont installés
- Vérifier que `.env.local` existe (sinon le créer depuis `.env.example`)
- Installer les dépendances si `node_modules` est absent
- Afficher un récapitulatif de l'environnement configuré

Exercice 2 — Analyse des logs

Écrire une commande pipeline pour analyser les logs Railway.

- Filtrer les lignes contenant `ERROR` ou `WARN`
- Extraire la date, le niveau et le message de chaque ligne
- Compter le nombre d'erreurs par heure
- Sauvegarder le rapport dans `errors_report.txt`

Exercice 3 — Script de smoke test

Créer un script `smoke_test.sh` complet pour Social Seller.

- Tester `/health` → 200
- Tester `/products` sans token → 401
- Tester le webhook WhatsApp verify
- Afficher un rapport final avec ✓/✗ pour chaque test
- Retourner exit code 1 si au moins un test échoue

Bibliographie & Ressources

Références

- tldp.org/LDP/abs/html — Advanced Bash-Scripting Guide (complet)
- explainshell.com — expliquer n'importe quelle commande shell
- shellcheck.net — linter pour scripts shell (détecter les bugs)

Outils

- tldr.sh — man pages simplifiées (exemples pratiques)
- crontab.guru — éditeur visuel pour les expressions cron
- stedolan.github.io/jq/manual — référence jq complète

Livres

- William Shotts — The Linux Command Line (nostarch.com, gratuit en ligne)
- Carl Albing — Bash Cookbook (O'Reilly)